

AULA 07 • PROGRAMAÇÃO ORIENTADA A OBJETOS

Herança, sobrescrita e polimorfismo

Reaproveitar código com `extends`, especializar com `@Override`
e deixar o Java escolher o comportamento certo em tempo
de execução.

Curso de Java • Prof. Rob Silva

O caminho de hoje.

- 01 Por que herança? O problema do código repetido
- 02 `extends` e `super`
- 03 `@Override` — sobrescrever um método
- 04 Polimorfismo (introdução)
- 05 Aplicando no sistema da MegaStore (JAVA-104)
- 06 Tarefa da aula — ticket JAVA-107

Do Java básico para a orientação a objetos.

- No `JAVA-104` vocês fizeram o sistema da MegaStore: menu com `Scanner`, CRUD de produtos e categorias e relatórios.
- Até agora, quase tudo numa estrutura só. Hoje organizamos com classes que se relacionam.
- Meta da aula: menos código repetido, mais código reaproveitado.

Herdar = reaproveitar e especializar.

- Uma **subclasse** ganha os atributos e métodos da **superclasse**.
- Relação "**é um**": Cachorro **é um** Animal.
- A subclasse pode adicionar coisas novas e mudar o que herdou.



A classe-base concentra o que é comum.

```
public class Animal {  
    protected String nome;  
    protected int idade;  
  
    public Animal(String nome, int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
    public void emitirSom() {  
        System.out.println(nome + " faz um som.");  
    }  
}
```

Tudo que **todo animal** tem mora aqui — escrito uma única vez. `protected` = visível para as subclasses.

A subclasse herda tudo com uma palavra.

```
public class Cachorro extends Animal {  
  
    public Cachorro(String nome, int idade) {  
        super(nome, idade);    // chama o construtor de Animal  
    }  
  
    // Cachorro já herda nome, idade, emitirSom() e dormir().  
    // Não precisamos reescrever nada disso.  
}
```

`extends Animal` diz "Cachorro **é um** Animal". O `super(...)` reaproveita o construtor da base.

@Override troca o comportamento herdado.

```
public class Cachorro extends Animal {
    @Override
    public void emitirSom() {
        System.out.println(nome + " faz: Au au!");
    }
}

public class Gato extends Animal {
    @Override
    public void emitirSom() {
        System.out.println(nome + " faz: Miau!");
    }
}
```

Mesmo nome de método, comportamento diferente. `@Override` avisa o Java: "estou reescrevendo um método da base".

super reaproveita a base e ainda adiciona.

```
public class Cachorro extends Animal {  
    @Override  
    public void emitirSom() {  
        super.emitirSom();    // mantém o comportamento de Animal  
        System.out.println(nome + " faz: Au au!");  
    }  
}
```

Nem sempre queremos jogar fora o que a base fazia. `super.metodo()` executa a versão herdada e acrescenta a sua.

POLIMORFISMO · INTRODUÇÃO

Um tipo. Muitos comportamentos.

A mesma chamada — `emitirSom()` — produz resultados diferentes dependendo do objeto real. Você programa pensando em `Animal`; o Java resolve o resto.

Uma lista de Animal, cada um do seu jeito.

```
Animal[] animais = {  
    new Cachorro("Rex", 3),  
    new Gato("Mimi", 2),  
    new Animal("Bicho", 1)  
};  
  
for (Animal a : animais) {  
    a.emitirSom();    // cada um responde do seu jeito  
}  
  
// Saída no console:  
// Rex faz: Au au!   | Mimi faz: Miau!   | Bicho faz um som.
```

O array é de `Animal`, mas dentro tem Cachorro, Gato e Animal. O `for` é um só — o **resultado muda sozinho**.

Trazendo a ideia para o nosso Produto.

```
public class Produto {  
    protected String nome;  
    protected double preco;  
    protected int quantidade;  
    protected String categoria;  
  
    public Produto(String nome, double preco, int quantidade, String categoria) {  
        this.nome = nome;  
        this.preco = preco;  
        this.quantidade = quantidade;  
        this.categoria = categoria;  
    }  
  
    public String descricao() {  
        return nome + " | R$ " + preco + " | Qtde: " + quantidade + " | " + categoria;  
    }  
}
```

É o `Produto` do seu sistema da MegaStore (`JAVA-104`) — agora como superclasse, com um `descricao()` que monta a linha do relatório.

Um tipo especial sem duplicar Produto.

```
public class ProdutoPerecivel extends Produto {  
    private String dataValidade;  
  
    public ProdutoPerecivel(String nome, double preco, int quantidade,  
                             String categoria, String dataValidade) {  
        super(nome, preco, quantidade, categoria);  
        this.dataValidade = dataValidade;  
    }  
  
    @Override  
    public String descricao() {  
        return super.descricao() + " | validade: " + dataValidade;  
    }  
}
```

Adiciona **um** campo, chama `super(...)` no construtor e estende o relatório com `super.descricao()`.

O relatório fica mais simples, não mais complexo.

```
// Opção 3 do menu: "Relatório de Estoque"
public static void relatorioEstoque(Produto[] produtos) {
    System.out.println("=== RELATÓRIO DE ESTOQUE ===");

    for (Produto p : produtos) {
        // p pode ser Produto, ProdutoPerecivel ou ProdutoEletronico
        System.out.println(p.descricao()); // o Java escolhe o certo
    }
}
```

Um único laço, **sem if por tipo**. É o **Relatório de Estoque** (opção 3) do `JAVA-104` — agora pronto para qualquer tipo de produto.

Três ideias para levar para casa.

extends

A subclasse herda atributos e métodos da superclasse. Reuso sem copiar e colar.

@Override

A subclasse troca o comportamento de um método herdado. `super.metodo()` mantém a base.

polimorfismo

Uma variável do tipo base aponta para objetos de subclasses. O método certo roda sozinho.

JAVA-107

MÉDIO

TO DO

BACKEND

RESPONSÁVEL: VOCÊ

Tipos de produto no estoque com herança

HISTÓRIA DO USUÁRIO

Como gerente de estoque da MegaStore, preciso diferenciar **produtos perecíveis** (validade) e **produtos eletrônicos** (garantia), com informações próprias, sem duplicar a classe `Produto` — para que o relatório mostre cada item corretamente.

CONTEXTO

Evolui o sistema do `JAVA-104` : hoje todo item usa a mesma classe `Produto` com a categoria em texto. Com o catálogo crescendo, a MegaStore precisa de tipos especializados, reaproveitando o que já existe.

JAVA-107

MÉDIO

PRONTO QUANDO TODOS OS ITENS PASSAREM

CRITÉRIOS DE ACEITE

- ☒ Produto é a superclasse com nome, preco, quantidade, categoria e o método descricao().
- ☒ ProdutoPerecivel extends Produto COM dataValidade e @Override em descricao().
- ☒ ProdutoEletronico extends Produto COM garantiaMeses e @Override em descricao().
- ☒ Construtores usam super(...); as subclasses usam super.descricao().
- ☒ O **Relatório de Estoque** (opção 3) percorre os produtos e chama descricao() — **sem if por tipo**. O menu e o Scanner do JAVA-104 continuam iguais.

DESAFIO EXTRA

Crie um terceiro tipo (ex.: ProdutoImportado com taxaImportacao) — o laço do relatório **não pode mudar**.

- #megastore
- #java-104
- #heranca
- #entrega: próxima aula

Use só o que vimos hoje: extends, @Override, super e polimorfismo.

